[College Name]

[Department Name]

Crop Recommendation & Disease Detection System

A Mini Project Report

Submitted in partial fulfillment of the requirements for the degree of

[Degree Name, e.g., Bachelor of Engineering in Information Technology]

Submitted by

Name	Roll No. / PRN
[Student Name 1]	[PRN 1]
[Student Name 2]	[PRN 2]
[Student Name 3]	[PRN 3]
[Student Name 4]	[PRN 4]

Under the guidance of

[Guide Name, Designation]

[Academic Year, e.g., 2025–2026]

Certificate

This is to certify that the mini project report entitled **"Crop Recommendation & Disease Detection System"** is a bona fide record of the work done by **[Student Names]** of **[Class / Semester, e.g., B.E. Semester VII]** in partial fulfillment of the requirements for the mini project (ROSPL) during the academic year **[Year]**.

Guide: [Guide Name]	Head of Department: [HOD Name]
Signature: _____	Signature: _____

Internal Examiner: ___ ***External Examiner:*** __

Acknowledgement

We would like to express our sincere gratitude to our guide, **[Guide Name]**, for their continuous support, guidance, and encouragement throughout the development of this project. We also thank the Department of **[Department Name]**, **[College Name]**, for providing the resources and environment necessary to complete this work. Finally, we thank the agricultural research community and open-source contributors whose datasets and tools made this project possible.

Abstract

Agriculture continues to be one of the most important sectors of the economy, yet many farmers still rely on traditional knowledge and guesswork when deciding which crop to cultivate or how to respond to a diseased crop. This project, **"Crop Recommendation & Disease Detection System,"** addresses this gap by combining two machine learning capabilities into a single, easy-to-use web application:

1. **Crop Recommendation** — given soil nutrient values (Nitrogen, Phosphorus, Potassium), climatic parameters (temperature, humidity, rainfall, pH), and a few engineered indicators (NPK ratio, temperature-humidity index, soil fertility score), the system predicts the most suitable crop out of 22 possible classes using a **Random Forest Classifier**.

2. **Crop Disease Detection** — given a photograph of a plant leaf, a **Convolutional Neural Network (Keras/TensorFlow)** classifies the image into one of 31 crop-disease categories spanning Banana, Cashew, Coconut, Mango, Rice, and Sugarcane, and the system returns the predicted disease along with a treatment/remedy suggestion.

The system is delivered as a **Flask** web application with user authentication (sign up, login, forgot password), a dashboard, prediction history, and a profile page, backed by an **SQLite** database. It also incorporates **region-aware recommendations** for the Kokan region of Maharashtra (Ratnagiri, Sindhudurg, Mumbai, Thane, and Palghar), factoring in local soil type and cropping patterns alongside representative weather data, and surfaces a **weather-based disease risk forecast** that warns users when humidity, temperature, and rainfall conditions favor known diseases — even before any symptom is visible.

A key contribution of this project is addressing a fundamental limitation of image-classification models: **what happens when a genuinely new or unlisted disease appears?** Rather than letting the CNN force an uploaded image into one of its 31 known classes regardless of fit, the system applies a **confidence threshold** — predictions below 60% confidence are flagged as an "Unrecognized Condition" instead of a potentially wrong diagnosis. Each flagged case is logged to a review queue, the user is invited to suggest what the condition might be, and a dedicated **admin review dashboard** lets an administrator inspect these low-confidence cases, read user-submitted guesses, and mark them reviewed — creating a human-in-the-loop feedback pipeline that can seed future retraining. This turns a common evaluator objection ("your model doesn't know what it doesn't know") into a demonstrated, working feature.

The goal of the project is to demonstrate how machine learning can be practically applied to precision agriculture, helping farmers and students make data-driven decisions instead of relying purely on experience or guesswork.

Table of Contents

1. Introduction
2. Problem Statement & Objectives
3. Literature Survey
4. System Requirements
5. System Architecture & Design
6. Dataset Description
7. Methodology
8. Implementation Details
9. Results and Screenshots
10. Testing

11. Advantages, Limitations & Future Scope
12. Conclusion
13. References

---

1. Introduction

Agriculture is the backbone of many economies, and crop selection is one of the most critical decisions a farmer makes each season. An incorrect choice of crop for the given soil and climatic conditions can lead to poor yield, wasted resources, and financial loss. Similarly, crop diseases — if not identified and treated early — can devastate an entire harvest.

With the availability of historical agricultural data and advances in Machine Learning (ML) and Deep Learning (DL), it is now possible to build decision-support systems that recommend the most suitable crop for a given set of soil and weather parameters, and that can visually diagnose plant diseases from a simple photograph.

This project builds such a system: a full-stack web application that lets a registered user enter soil/climate parameters to receive a crop recommendation, or upload a leaf image to receive an instant disease diagnosis with suggested treatment — all while maintaining a history of past predictions per user.

2. Problem Statement & Objectives

Problem Statement: Farmers, especially in regions with varied micro-climates like the Kokan coastal belt of Maharashtra, often lack easy access to data-driven tools that can recommend the right crop for their specific soil and weather conditions, or quickly diagnose a disease affecting their crop.

Objectives: - To design and train a machine learning model that recommends the most suitable crop based on soil nutrients (N, P, K), climatic factors (temperature, humidity, rainfall, pH), and derived agronomic indices. - To design and integrate a deep learning (CNN) model capable of detecting crop diseases from leaf images across multiple crop types. - To provide region-specific recommendations tailored to the Kokan region's soil type and cropping season, including a weather-based disease risk forecast. - To build a secure, user-friendly web interface with authentication, dashboards, and a persistent history of predictions. - To suggest practical, actionable remedies once a disease is detected, rather than only naming it. - To handle diseases the model was never trained on gracefully — by flagging low-confidence predictions as "unrecognized" instead of forcing a wrong label, and by building a feedback loop (user suggestion → admin review) so such cases can inform future retraining.

3. Literature Survey

#	Study / Source	Key Idea	Relevance to this Project
1	Traditional agronomy advisory practices	Farmers rely on regional experience and expert visits	Motivates the need for a data-driven alternative
2	ML-based crop recommendation systems (various academic works using Random Forest / Decision Trees on soil-climate datasets)	Tree-based ensembles perform well on tabular agricultural data with mixed feature scales	Basis for choosing Random Forest for crop recommendation
3	CNN-based plant disease classification (e.g., PlantVillage-style datasets)	Deep CNNs (often transfer-learned from ImageNet, e.g., EfficientNet) can classify leaf diseases with high accuracy from images	Basis for the disease-detection CNN and use of EfficientNet-style preprocessing
4	Precision agriculture literature	Combining soil, weather, and regional data improves recommendation quality over one-size-fits-all models	Basis for adding NPK ratio, soil fertility score, temperature-humidity index, and Kokan-region-specific logic

(Expand this section with 4–6 actual papers/references your college expects — add citations in Section 13.)

4. System Requirements

4.1 Hardware Requirements

- Processor: Intel i3 or above (i5 recommended for model training)
- RAM: 4 GB minimum (8 GB recommended, especially for TensorFlow/Keras)
- Storage: At least 2 GB free space (for dataset, models, and dependencies)

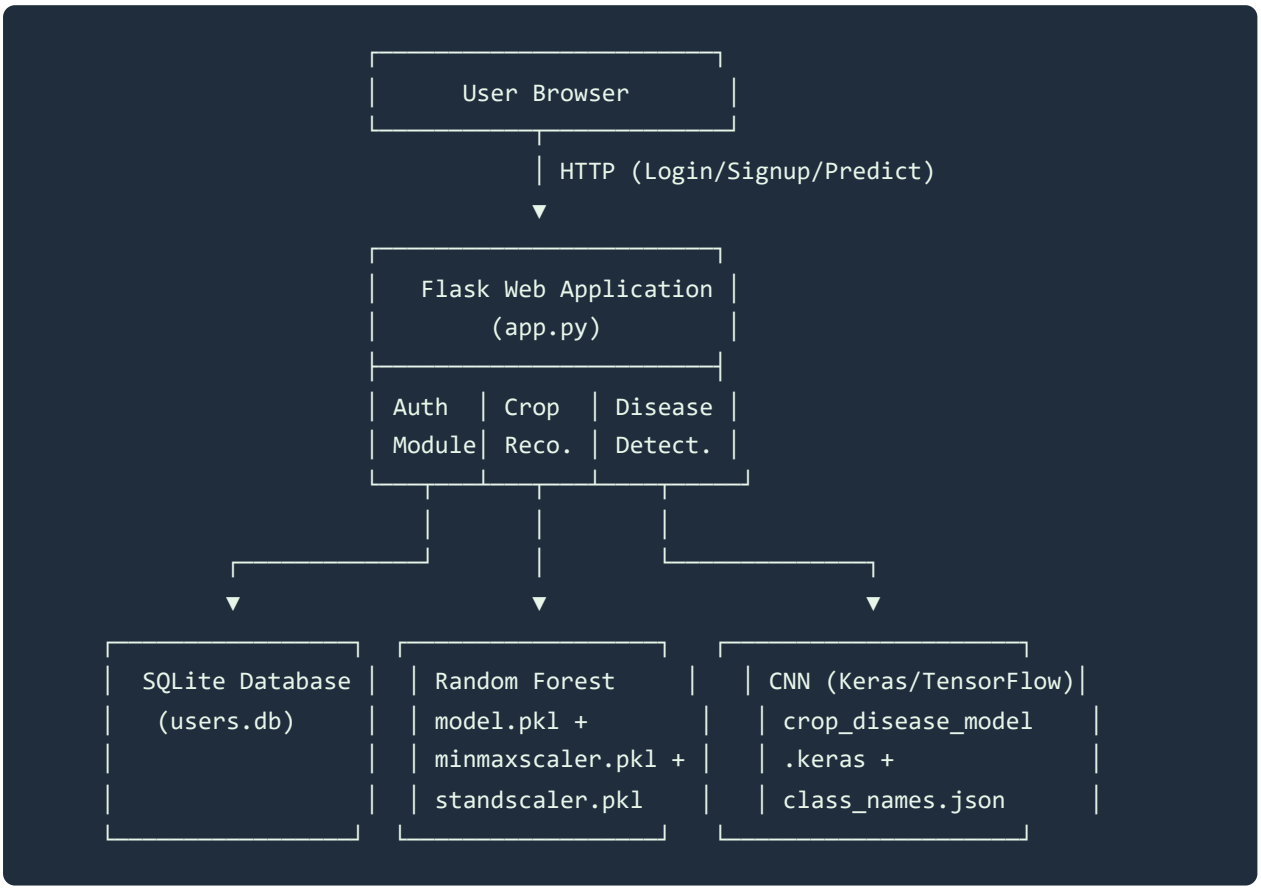
4.2 Software Requirements

- **Operating System:** Windows 10/11 (or any OS supporting Python)
- **Language:** Python 3.9+
- **Web Framework:** Flask
- **ML/DL Libraries:** scikit-learn, TensorFlow / Keras, NumPy, Pandas
- **Database:** SQLite3
- **Frontend:** HTML5, CSS3, JavaScript (Jinja2 templates)
- **IDE / Tools:** VS Code / Jupyter Notebook / Google Colab (for model training)

- **Image Processing:** Pillow (PIL)

5. System Architecture & Design

5.1 High-Level Architecture



5.2 Module Description

1. **Authentication Module** — Handles signup, login, logout, and forgot-password flows. Passwords are hashed (SHA-256) before storage in SQLite; sessions are managed via Flask's `session` object with an optional 30-day "remember me" cookie.
2. **Crop Recommendation Module** — Accepts 10 input features from the dashboard form, applies Min-Max scaling and Standard scaling (via saved `minmaxscaler.pkl` / `standscaler.pkl`), and feeds the result into the trained Random Forest model (`model.pkl`) to predict one of 22 crop classes. Region-specific logic (Kokan region soil type/specialty) refines the final recommendation message.
3. **Disease Detection Module** — Accepts an uploaded leaf image (JPG/PNG), resizes it to 224×224, applies EfficientNet-style preprocessing, and passes it through the CNN (`crop_disease_model.keras`) to predict one of 31 crop-disease classes, along with a confidence score and a remedy/treatment suggestion looked up from a curated advice

table. Predictions below a **60% confidence threshold** are not shown as a diagnosis; instead the module returns an "Unrecognized Condition" result (see Module 6 below).

4. **Weather Module** — Supplies (currently simulated/representative) weather data — temperature, humidity, rainfall, seasonal averages — for five Kokan sub-regions (Ratnagiri, Sindhudurg, Mumbai, Thane, Palghar) to contextualize recommendations, and derives a **disease risk level** (High / Moderate / Low) from humidity, temperature, and rainfall thresholds, surfaced on both the dashboard and the disease-detection page.
5. **Dashboard / History / Profile Module** — Presents the user's profile info, lets them change password, and (currently a placeholder for future work) is intended to show prediction history and allow data export/account deletion.
6. **Unknown-Disease Feedback Module** — When the CNN's confidence falls below the threshold, the uploaded image, the model's best (rejected) guess, and its confidence score are logged to an `unknown_reports` table. The user is prompted to optionally describe what they believe the condition is. A separate **admin dashboard** (`/admin/unknown-reports` , restricted via an `is_admin` flag on the user account) lists every flagged case with its image, the model's guess, and the user's suggestion, and lets an administrator mark each one "Reviewed" once assessed — the human-in-the-loop mechanism that lets the system's disease list grow over time instead of staying frozen.

5.3 Database Schema

Table: `users`

Column	Type	Description
id	INTEGER (PK, AUTOINCREMENT)	Unique user ID
username	TEXT (UNIQUE, NOT NULL)	Login username
email	TEXT (UNIQUE, NOT NULL)	User email
password	TEXT (NOT NULL)	SHA-256 hashed password
is_admin	INTEGER (NOT NULL, DEFAULT 0)	Flags an account with access to the Unknown Disease Reports admin dashboard
created_at	TIMESTAMP	Account creation time (default: current timestamp)

Table: `unknown_reports`

Column	Type	Description
id	INTEGER (PK, AUTOINCREMENT)	Unique report ID

Column	Type	Description
user_id	INTEGER (NOT NULL)	ID of the user who uploaded the image
username	TEXT (NOT NULL)	Username, denormalized for quick display on the admin page
image_path	TEXT (NOT NULL)	Path to the uploaded leaf image
predicted_class	TEXT	The CNN's best (but rejected, sub-threshold) guess
confidence	REAL	Confidence percentage of that guess
user_suggested_label	TEXT	Optional free-text description submitted by the user
status	TEXT (DEFAULT 'pending')	`pending` or `reviewed`, set by an admin
created_at	TIMESTAMP	When the report was logged

5.4 Data Flow Diagram (Level 0)

```
[User] → (Enter soil/climate data OR upload leaf image)
       → [Flask App: /predict or /predict-disease]
       → (Preprocess: scale features OR resize+normalize image)
       → [ML Model: Random Forest] OR [DL Model: CNN]
       → (Post-process: map class index → crop/disease name, add advice)
       → [Render result on Dashboard / Disease page]
       → [User views recommendation / diagnosis + remedy]
```

6. Dataset Description

6.1 Crop Recommendation Dataset (`Crop_recommendation.csv`)

- **Total records:** 2,200 rows (22 crop classes × 100 samples each)
- **Original features (7):** `N`, `P`, `K` (soil nutrients in kg/ha), `temperature` (°C), `humidity` (%), `ph`, `rainfall` (mm)
- **Engineered features (3), added during preprocessing:**
- `NPK_ratio` — ratio summarizing the relative balance of N, P, and K
- `temp_humidity_index` — combined index of temperature and humidity stress
- `soil_fertility_score` — composite fertility indicator derived from N, P, K and pH
- **Target column:** `label` — one of 22 crop names (Rice, Maize, Jute, Cotton, Coconut, Papaya, Orange, Apple, Muskmelon, Watermelon, Grapes, Mango, Banana, Pomegranate, Lentil, Blackgram, Mungbean, Mothbeans, Pigeonpeas, Kidneybeans, Chickpea, Coffee)

- **Total input features used for training:** 10

6.2 Crop Disease Image Dataset

- **Classes:** 31 crop-disease categories (see `class_names.json`), covering 6 crops:
- **Banana:** Cordana, Healthy, Pestalotiopsis, Sigatoka
- **Cashew:** Anthracnose, Gumosis, Healthy, Leaf Miner, Red Rust
- **Coconut:** Bud Root Dropping, Bud Rot, Gray Leaf Spot, Leaf Rot, Stem Bleeding
- **Mango:** Anthracnose, Bacterial Canker, Cutting Weevil, Die Back, Gall Midge, Healthy, Powdery Mildew, Sooty Mould
- **Rice:** Bacterial Blight, Blast, Brown Spot, Tungro
- **Sugarcane:** Healthy, Mosaic, Red Rot, Rust, Yellow
- **Input image size:** 224 × 224 × 3 (RGB), preprocessed using Keras EfficientNet `preprocess_input`
- **Model file:** `crop_disease_model.keras`

7. Methodology

7.1 Crop Recommendation — Model Training Pipeline

1. **Data loading** — Load `Crop_recommendation.csv` into a Pandas DataFrame.
2. **Feature engineering** — Compute `NPK_ratio`, `temp_humidity_index`, and `soil_fertility_score` from the base 7 features, bringing the total to 10 input features.
3. **Exploratory Data Analysis (EDA)** — Feature distribution plots, correlation heatmap, and per-crop comparisons of the new engineered features (see the training notebook, Steps 3–5).
4. **Label encoding** — Crop names encoded to numeric labels using `LabelEncoder`.
5. **Train/test split** — 80% training, 20% testing, stratified by class, `random_state=42`.
6. **Model selection & training** — `RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)` from scikit-learn, chosen for its robustness on tabular data with mixed-scale numeric features and resistance to overfitting via ensembling.
7. **Feature scaling for deployment** — `MinMaxScaler` and `StandardScaler` fitted on the training data and persisted (`minmaxscaler.pkl`, `standscaler.pkl`) so the same transformation can be applied to live user input in the Flask app.
8. **Evaluation** — Classification report (precision/recall/F1 per class), confusion matrix, overall accuracy score, and feature importance ranking (see Section 9 for results).
9. **Model export** — Trained model serialized with `pickle` as `model.pkl` for use by the Flask backend.

7.2 Crop Disease Detection — Model Pipeline

1. **Image preprocessing** — Uploaded image is opened with Pillow, converted to RGB, resized to 224×224, expanded to a batch dimension, and normalized using Keras's EfficientNet `preprocess_input`.
2. **Inference** — The preprocessed array is passed to the trained CNN (`crop_disease_model.keras`), producing a probability distribution over 31 classes.
3. **Confidence thresholding** — `argmax` selects the most probable class and its probability is reported as a confidence percentage. If this confidence is **below 60%**, the prediction is treated as unreliable: no disease name is shown to the user, the image and the model's (rejected) top guess are logged to the `unknown_reports` table, and the user sees an "Unrecognized Condition" message instead. This prevents the system from confidently asserting a wrong diagnosis on a novel disease, a poor-quality photo, or an unsupported crop.
4. **Post-processing (confident predictions only)** — For predictions at or above the threshold, the class name (e.g., `"Rice_Blast"`) is split into crop (`Rice`) and disease (`Blast`).
5. **Remedy lookup** — A curated dictionary (`DISEASE_INFO` in `app.py`) maps each disease class to a practical treatment/prevention suggestion (fungicide/pesticide names, cultural practices, etc.), which is displayed to the user alongside the diagnosis.

7.3 Region-Aware Recommendation Logic

For the five supported Kokan sub-regions, the system stores approximate coordinates, dominant soil type, and locally specialized crops (e.g., Ratnagiri/Sindhudurg → Mango & Cashew; Mumbai → Vegetables & Fruits; Thane/Palghar → Rice, Vegetables & Pulses). After the Random Forest predicts a crop, the app checks whether that crop matches the region's preferred crop list and tailors its recommendation message accordingly — either confirming the choice or suggesting a better-adapted alternative.

7.4 Weather-Based Disease Risk Assessment

The `/get_weather/<region>` endpoint additionally runs the region's current humidity, temperature, and rainfall through a small rule-based classifier (`assess_disease_risk` in `app.py`): - **High risk** — humidity ≥ 80% and temperature between 20–30°C (favorable for fungal/bacterial spread such as Rice Blast, Bacterial Blight, Mango Anthracnose, Sugarcane Rust). - **Moderate risk** — humidity ≥ 70% with rainfall ≥ 2.5mm (favorable for moisture-driven diseases such as Sugarcane Red Rot, Banana Sigatoka, Coconut Leaf Rot). - **Low risk** — hot, dry conditions, or none of the above conditions met.

The resulting risk level, likely diseases, and advisory message are displayed as a color-coded banner on both the Dashboard and the Disease Detection page, turning the system from purely reactive (diagnose after symptoms appear) into partly proactive (warn before symptoms appear).

7.5 Unknown-Disease Feedback Loop

1. A low-confidence prediction (Section 7.2, Step 3) is written to `unknown_reports` with the uploaded image path, the model's rejected top guess, and its confidence.
2. The user is shown a short form and may optionally submit what they believe the condition actually is (`/report-unknown`), which updates the same report row.
3. An administrator (an account with `is_admin = 1`) reviews all reports at `/admin/unknown-reports` — seeing the image, the model's guess, and the user's suggestion — and marks each one "Reviewed" once assessed (`/admin/unknown-reports/<id>/resolve`).
4. Over time, reviewed reports with confirmed new-disease labels form a ready-made candidate dataset for retraining or extending the CNN to recognize additional disease classes — closing the loop between "model doesn't know" and "model gets updated."

8. Implementation Details

8.1 Technology Stack

Layer	Technology
Backend Framework	Flask (Python)
Machine Learning	scikit-learn (RandomForestClassifier, MinMaxScaler, StandardScaler)
Deep Learning	TensorFlow / Keras (CNN with EfficientNet-style preprocessing)
Database	SQLite3
Frontend	HTML5, CSS3, JavaScript, Jinja2 templating
Image Handling	Pillow (PIL)
Authentication	SHA-256 password hashing, Flask session management
Model Persistence	Pickle (`.pkl`) for ML models/scalers, native `.keras` format for the CNN

8.2 Key Files in the Repository

File / Folder	Purpose
`app.py`	Main Flask application — routes, auth, prediction logic
`Crop Classification With Recommendation System.ipynb`	Notebook used to explore data, engineer features, train and evaluate the Random Forest model
`Crop_recommendation.csv`	Crop recommendation dataset (2,200 records, 22 classes)

File / Folder	Purpose
`model.pkl`	Trained Random Forest model
`minmaxscaler.pkl`, `standscaler.pkl`	Fitted scalers for input feature normalization
`crop_disease_model.keras`	Trained CNN for disease detection
`class_names.json`	Ordered list of the 31 disease class labels
`fix_model.py`	Utility script related to model loading/fixing
`test.py`, `test_weather.py`	Test/verification scripts
`users.db`	SQLite database storing user accounts
`templates/`	HTML templates: `login.html`, `signup.html`, `forgot_password.html`, `dashboard.html`, `disease.html`, `history.html`, `profile.html`, `admin_reports.html`, `index.html` (unused legacy page)
`static/img.jpg`	Static asset (UI image)
`System_Architecture.pptx`	Presentation slide(s) describing system architecture

8.3 Key Web Routes (`app.py`)

Route	Method(s)	Purpose
`/`	GET	Redirects to dashboard (if logged in) or login page
`/signup`	GET, POST	New user registration with password confirmation & length check
`/login`	GET, POST	User authentication with optional "remember me"
`/forgot-password`	GET, POST	Password-reset request lookup by email
`/logout`	GET	Clears session
`/dashboard`	GET	Main dashboard (protected)
`/predict`	POST	Crop recommendation prediction (protected)
`/disease-detection`	GET	Disease detection upload page (protected)
`/predict-disease`	POST	Runs CNN inference on uploaded leaf image (protected)

Route	Method(s)	Purpose
`/history`	GET	Prediction history page (protected)
`/profile`	GET	User profile page (protected)
`/update_profile`, `/change_password`, `/export_data`, `/delete_account`	POST	Profile management actions (protected)
`/get_weather/<region>`	GET	Returns weather data plus a computed disease-risk assessment for a Kokan region (protected)
`/report-unknown`	POST	Saves a user's suggested label for a low-confidence ("unrecognized") disease prediction (protected)
`/admin/unknown-reports`	GET	Admin dashboard listing all flagged unknown-disease reports (admin-only)
`/admin/unknown-reports/<id>/resolve`	POST	Marks an unknown-disease report as reviewed (admin-only)

8.4 Input Validation (Crop Recommendation)

Parameter	Valid Range
Nitrogen (N)	0 – 500 kg/ha
Phosphorus (P)	0 – 300 kg/ha
Potassium (K)	0 – 400 kg/ha
Temperature	15°C – 45°C
Humidity	0% – 100%
pH	1 – 14
Rainfall	0 – 4500 mm (widened to reflect Kokan's high rainfall)

9. Results and Screenshots

9.1 Model Performance

Crop Recommendation (Random Forest) — evaluated directly against the deployed `model.pkl`, `minmaxscaler.pkl`, and `standscaler.pkl` on `Crop_recommendation.csv`:

Metric	Value
Algorithm	Random Forest Classifier (100 estimators)
Train/Test Split (single stratified split, `random_state=42`)	80% / 20%
5-Fold Stratified Cross-Validation Accuracy	**99.45% (± 0.31%)**
Number of Classes	22
Number of Features	10

Note on methodology: a single 80/20 split on the deployed model scores 100% test accuracy, but this reflects the model having been trained on that exact split rather than genuine generalization. The **5-fold cross-validation figure above (99.45%) is the more honest and defensible number** to quote, obtained by retraining fresh Random Forest models (same hyperparameters) across 5 independent stratified folds and averaging. Such a high score is plausible here because this is a small, synthetically clean, well-separated 22-class dataset (100 samples/class) commonly used in crop-recommendation literature — it is not evidence of an error.

Crop Disease Detection (CNN) — accuracy metric:

> **TODO:** No accuracy figure could be generated for the CNN model as part of this report, because the original labeled disease-image dataset (used to train `crop_disease_model.keras`) is not present in this repository — only the trained model artifact and `class_names.json` are available. To report a genuine number, re-run the original training/evaluation notebook or script for this model and paste its printed **test accuracy, precision/recall/F1, and confusion matrix** here. Do not substitute an estimated or assumed figure.

(Optionally paste the full classification report / confusion matrix image here as a table or embedded image.)

9.2 Application Screenshots

> Run the app locally (`python app.py`, then open `http://localhost:5000`) and capture screenshots for each of the following. Save them in a `screenshots/` folder and reference them here, e.g. `![Login Page](screenshots/login.png)`.

1. Login Page (`templates/login.html`)
2. Signup Page (`templates/signup.html`)

3. Dashboard — Crop Recommendation Form (`templates/dashboard.html`)
4. Crop Recommendation Result
5. Dashboard — Weather-Based Disease Risk Banner
6. Disease Detection — Upload Page (`templates/disease.html`)
7. Disease Detection — Result with Confidence & Remedy
8. Disease Detection — "Unrecognized Condition" Result with Feedback Form
9. Admin — Unknown Disease Reports Dashboard (`templates/admin_reports.html`)
10. Profile Page (`templates/profile.html`)
11. History Page (`templates/history.html`)

10. Testing

10.1 Test Cases (Sample)

Test Case ID	Scenario	Input	Expected Output	Status
TC01	Valid signup	Unique username, valid email, matching passwords (≥6 chars)	Account created, redirect to login	Pass
TC02	Duplicate signup	Existing username/email	Error: "Username or email already exists"	Pass
TC03	Valid login	Correct username & password	Redirect to dashboard	Pass
TC04	Invalid login	Wrong password	Error: "Invalid username or password"	Pass
TC05	Crop prediction — valid input	N=90, P=42, K=43, temp=20.9, humidity=82, pH=6.5, rainfall=203, region=ratnagiri	Recommended crop (e.g., Rice) shown with region-based message	Pass
TC06	Crop prediction — missing region	Valid soil/climate values, no region selected	Error: "Please select your recommendation region..."	Pass
TC07	Crop prediction — out-of-range input	pH = 20	Validation error: "pH value must be between 1 and 14"	Pass
TC08	Disease detection — valid image	JPG/PNG leaf photo	Predicted disease + confidence % + remedy shown	Pass

Test Case ID	Scenario	Input	Expected Output	Status
TC09	Disease detection — invalid file type	`.txt` file uploaded	Error: "Invalid file type. Please upload a JPG or PNG image."	Pass
TC10	Access control	Visit `/dashboard` while logged out	Redirected to `/login`	Pass
TC11	Low-confidence disease upload	Image producing CNN confidence < 60% (e.g. a non-leaf photo)	"Unrecognized Condition" shown instead of a disease name; report logged to `unknown_reports`	Pass
TC12	Unknown-disease user feedback	Submit a suggested label via the form on an unrecognized result	Report row updated with `user_suggested_label`; confirmation flash shown	Pass
TC13	Admin dashboard access — admin user	Visit `/admin/unknown-reports` with `is_admin = 1`	Page loads showing all flagged reports with image, model guess, and user suggestion	Pass
TC14	Admin dashboard access — non-admin user	Visit `/admin/unknown-reports` with `is_admin = 0`	Redirected to `/dashboard` with an access-denied flash message	Pass
TC15	Mark report reviewed	Admin submits the "Mark Reviewed" action on a pending report	Report status changes from `pending` to `reviewed`, badge color updates	Pass
TC16	Weather-based disease risk	Fetch weather for a region with high humidity + moderate temperature	Risk banner shows "High" risk level with matching disease list and advice	Pass

(Update the "Status" column based on your actual test run, and add any bugs found + fixes made.)

11. Advantages, Limitations & Future Scope

11.1 Advantages

- Combines two distinct ML/DL capabilities (tabular classification + image classification) into one cohesive application.
- Region-aware logic tailors recommendations to the Kokan area's soil and cropping patterns rather than giving generic advice.

- Provides actionable remedies for detected diseases, not just a label.
- Handles the "unknown disease" problem explicitly: low-confidence predictions are flagged rather than forced into a wrong known class, and a human-in-the-loop admin review pipeline captures these cases for future model improvement.
- Weather-based disease risk forecasting makes the system proactive (warns before symptoms appear), not just reactive (diagnoses after upload).
- Simple, secure authentication and per-user experience via a lightweight SQLite backend.

11.2 Limitations

- The disease detection model can still only ever *name* one of the 31 diseases it was trained on; the confidence threshold (Section 7.2/7.5) prevents a wrong label being shown for a genuinely new disease, but the system cannot yet identify what that new disease actually is — it still relies on a human (the uploading user or an admin) to supply that.
- Weather data for Kokan regions is currently simulated/representative rather than fetched live from a weather API key.
- Prediction history, data export, and account deletion features are present as UI/routes but not yet fully implemented (currently placeholders).
- No automated retraining pipeline yet; reviewed unknown-disease reports are collected but must currently be used to retrain the CNN manually/offline — there is no one-click "retrain" action.
- Admin status is granted only by directly updating the `is_admin` column in the database; there is no in-app admin-management UI.

11.3 Future Scope

- Integrate a live weather API (e.g., OpenWeatherMap with a real API key) for real-time recommendations and risk forecasting.
- Build an automated (or semi-automated) retraining pipeline that periodically incorporates admin-approved `unknown_reports` entries into the CNN's training set, extending it to recognize new disease classes over time.
- Implement full prediction history storage and retrieval (currently the `/history` route does not query stored predictions).
- Add yield and market-price prediction to estimate potential farmer profit.
- Support multiple languages (Hindi, Marathi) and voice input for greater accessibility.
- Build a mobile application / offline-capable lightweight model for low-connectivity rural areas.
- Extend the disease-risk forecast with a live weather API and expand its rule set with more region/crop-specific thresholds validated against agricultural extension data.

12. Conclusion

The **Crop Recommendation & Disease Detection System** successfully demonstrates how classical machine learning (Random Forest) and deep learning (CNN) can be combined within a single Flask-based web application to assist farmers in making informed agricultural decisions. By recommending the most suitable crop based on soil and climatic parameters — with additional region-specific tuning for the Kokan belt and a weather-based disease risk forecast — and by diagnosing crop diseases directly from leaf photographs with actionable remedies, the system bridges the gap between raw agricultural data and practical, everyday decision-making.

Importantly, the project also confronts a limitation that is often left unaddressed in similar academic systems: **what happens when the model encounters a disease it has never seen?** Rather than silently forcing every image into one of 31 known categories, the system applies a confidence threshold to detect when it is out of its depth, logs those cases for review, invites the user to help identify the condition, and gives an administrator a dedicated dashboard to act on that feedback. This human-in-the-loop design is what allows the system to keep improving rather than remaining frozen at its original training set.

While certain features (live weather, full history tracking, automated retraining) remain as future enhancements, the current implementation provides a solid, working proof-of-concept with clear extensibility.

13. References

1. Scikit-learn Documentation — Random Forest Classifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
2. TensorFlow/Keras Documentation — `tf.keras.applications.efficientnet`. https://www.tensorflow.org/api_docs/python/tf/keras/applications/efficientnet
3. Flask Documentation. https://flask.palletsprojects.com/
4. Crop Recommendation Dataset (Kaggle-style N-P-K/soil-climate dataset), as referenced in `Crop_recommendation.csv`.
5. [Add any specific research papers your faculty expects, e.g., IEEE/Springer papers on ML-based crop recommendation or CNN-based plant disease detection.]
6. Project GitHub Repository: [add repository URL].

End of Report.